NetDoctor

NETWORK DIAGNOSTIC SYSTEM — PHASE 1 COMPLETION REPORT

Project Phase: Phase 1 (UI & Project Foundation)	**Status:** Completed & Verified
Master Spec: docs/PRD.md	**Architecture:** docs/ARCHITECTURE.md
Test Coverage: 6/6 Vitest Tests Passed (100%)	**Dev Server:** http://localhost:5173

1. Executive Summary & Product Identity

NetDoctor is a structured network diagnostic and troubleshooting system that converts natural-language problem descriptions into an adaptive, evidence-based troubleshooting workflow. NetDoctor is engineered specifically as a dedicated diagnostic system rather than a generic conversational chatbot.

Phase 1 has established the complete frontend foundation and professional UI structure. The application strictly embodies the **Clean Network Dashboard** aesthetic, enforcing strict design constraints: zero emojis, zero stickers, zero AI branding, zero confidence scores, and high-contrast professional typography.

2. Design System Tokens & Brand Compliance

Element	Token Role	Hex Code	Compliance Status
Page Background	Light Canvas	`#F7F9FC`	Active & Verified
Card Surfaces	White Panels	`#FFFFFF`	Active & Verified
Primary Text	Dark Navy	`#172033`	Active & Verified
Secondary Text	Slate Gray	`#5B6577`	Active & Verified
Primary Action	Network Blue	`#2563EB`	Active & Verified
Secondary Accent	Teal Accent	`#0F9D9A`	Active & Verified
Borders / Dividers	Light Blue-Gray	`#E2E8F0`	Active & Verified

3. Implemented UI Screens & Components

A. Home Screen (Landing View):

- **Hero Header:** Clean NetDoctor brand title and 'Network Diagnostic System' subheader with live status badge.
- **Natural Language Input:** Large, responsive prompt box accepting user-described network issues.
- **OS Selector:** Radio pill buttons for Windows, macOS, Linux, and Other.
- **5 Common Problem Presets:** Clickable cards for Wi-Fi disconnection, no internet, slow speeds, DNS errors, and packet loss.
- **Network Pipeline Visual:** Subtle SVG topology showing Device → Gateway → DNS → Internet.

B. Diagnostic Workspace (Two-Panel Structured Layout):

- **Left Panel — Diagnostic Roadmap:** Sticky sidebar displaying step sequence, progress percentage bar, and visual states (Completed ✓, Current →, Pending ■, Failed ✗, Skipped -).
- **Right Panel — Active Step Action Center:** Step objective header, category tag, and beginner-friendly OS instructions.
- **Monospace Command Box:** Terminal container with a 1-click 'Copy' button and animated 'Copied!' feedback.
- **Expected Result Card:** Explanation and sample output of expected healthy network responses.
- **Result Submission Interface:** Tabbed layout supporting Quick Buttons, Raw CLI Output paste, and Screenshot dropzone.
- **'Why this step?' Drawer:** Detailed modal explaining the engineering purpose and isolated network layer without losing session progress.
- **Confirmation Modals:** Reversible dialogs for 'Stop Diagnosis' and 'Start New Diagnosis'.

4. Testing & Verification Summary

Test Case Scope	Target Element	Status
Brand & No-AI Compliance	Zero AI phrases, no probability ratios, exact titles	**PASSED**
OS Selector & Presets	Windows/macOS/Linux selector, 5 problem presets	**PASSED**
Workspace Transition	Launching diagnosis from input or preset card	**PASSED**
Step Navigation & Backtracking	Next Step, Previous Step, Skip Step controls	**PASSED**
'Why this step?' Modal	Technical drawer open/close without progress loss	**PASSED**
Session Controls & Modals	Stop Diagnosis & Start Over confirmation flows	**PASSED**
TypeScript Production Build	tsc -b && vite build (0 errors, 0 warnings)	**PASSED**

5. Decoupled Architecture & Phase 2 Preparation

All demonstration data in Phase 1 has been strictly isolated inside client/src/data/mockDiagnosticData.ts. The frontend state management in DiagnosticContext.tsx is architected to seamlessly connect to the backend REST API in subsequent phases.

Next Milestone — Phase 2 (Networking Problem Detection & Validation):

- Implement Problem Relevance Validator (networking vs. non-networking rejection).
- Implement Fuzzy Lexicon Matcher for typo tolerance and informal phrasing.
- Implement Vague Query Detector with single targeted follow-up questions.
- Connect OS enforcement to custom CLI instruction generator.